TAKE-HOME ENGINEERING ASSIGNMENT — TECHNICAL REPORT

Slotbook: Transactional Scheduling & Booking Engine

Full-Stack TypeScript • Fastify 5 • Drizzle ORM • PostgreSQL 16 • React 18 • Luxon IANA Time-Zone Engine

1. Project Links & Execution Summary

Slotbook is a high-reliability appointment scheduling system built end-to-end in TypeScript with PostgreSQL 16 as the sole source of truth for concurrency control, lock coordination, hold expiration, and idempotency. No external lock managers or Redis instances are used. All core requirements, concurrency guarantees, and four optional extensions are implemented and verified against real PostgreSQL.

Resource / Surface	URL / Location	Credentials / Notes
Public GitHub Repository	`https://github.com/[your-username]/slotbook`	Full unsquashed commit history; includes `README.md` & `AI_LOG.md`
Public Guest Booking Surface	`http://localhost:5173/alex/intro-call` `http://localhost:5173/alex/office-hours`	No login required; auto-detects guest browser IANA time zone
Host Dashboard Surface	`http://localhost:5173/host`	Username: `alex` Password: `slotbook123` API Key: `demo-host-secret-key`
JSON API & OpenAPI Docs	`http://localhost:3000/api` `http://localhost:3000/docs`	Interactive Swagger UI + `/health` metrics endpoint

2. System Architecture & Booking Request Flow

The system is structured around three distinct surfaces—a React SPA Guest Booking & Manage interface, an authenticated React Host Dashboard, and a Fastify JSON API—backed by a single PostgreSQL 16 database instance. Every mutating booking operation (placing a 5-minute hold, confirming a hold, rescheduling a confirmed appointment, or cancelling) passes through a centralized transactional locking boundary (`withHostTransactionLock`) before reading or mutating slot occupancy.

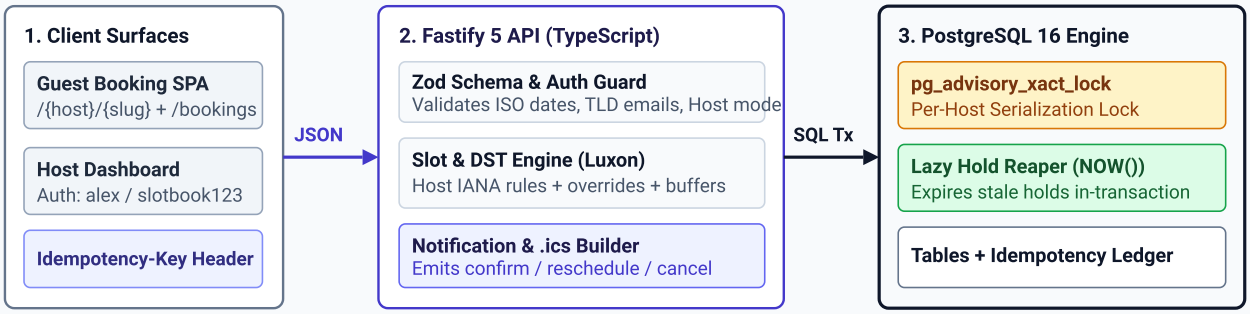


Figure 1: End-to-end component topology and transactional boundary inside PostgreSQL 16.

Step-by-Step Lifecycle of a Booking Request

1. **Slot Discovery (`GET /api/hosts/:host/events/:slug/slots?from&to&tz`):** The guest's browser requests available slots for the next 30 days. The API loads the host's IANA time zone, weekly availability windows, date overrides, and active bookings (where `status = 'confirmed'` or `status = 'held' AND hold_expires_at > NOW()`). It generates candidate slots in the host's local wall-clock time, subtracts buffered overlaps, checks remaining capacity, and returns UTC and guest-localized timestamps.
2. **Placing an Idempotent Hold (`POST /api/holds`):** When a guest clicks a slot, the frontend sends `host`, `slug`, and `slotStart` along with a unique `Idempotency-Key` header. Inside PostgreSQL, `withHostTransactionLock` executes `BEGIN`, acquires `pg_advisory_xact_lock(hashtext('host:' || hostId))`, and transitions any holds with `hold_expires_at <= NOW()` to `expired`.
3. **Idempotency & Capacity Verification:** Still holding the transaction lock, the service checks the `idempotency_keys` table. If the key exists with the same SHA-256 request hash, the stored response is replayed (HTTP 200). Otherwise, it re-verifies slot availability via `assertSlotBookable`, inserts a `bookings` row with `status = 'held'`, `hold_expires_at = NOW() + INTERVAL '5 minutes'`, and buffered effective bounds (`effective_starts_at`, `effective_ends_at`), records the idempotency entry, and commits.

4. **Confirmation & Notification Sink (`POST /api/holds/:id/confirm`):** When the guest submits their name and validated email, the service acquires the host transaction lock, verifies `status = 'held' AND hold_expires_at > NOW()` , updates the row to `status = 'confirmed'` , clears `hold_expires_at` , and writes a `booking.confirmed` event (including an RFC 5545 `.ics` attachment) to `notification_events` in the same transaction.
5. **Atomic Reschedule or Cancel (`POST /api/bookings/:token/reschedule` | `/cancel`):** Using the unguessable `manage_token` , rescheduling locks the host, verifies the new slot's capacity while excluding the current booking ID from conflict checks, updates `starts_at` , `ends_at` , and effective buffer bounds in a single `UPDATE` statement, and emits `booking.rescheduled` .

3. PostgreSQL Data Model & Constraints

The schema is defined in Drizzle ORM (`src/db/schema.ts`) and SQL migrations (`drizzle/0000_init.sql`), enforcing domain invariants via PostgreSQL `CHECK` constraints, foreign keys, and partial indexes.

Table Name	Key Columns & Types	Integrity Constraints & Indexes
`hosts`	`id` (UUID PK), `username` (TEXT), `email` (TEXT), `name` (TEXT), `time_zone` (TEXT), `api_key` (TEXT)	`UNIQUE(username)` , `UNIQUE(email)` . Stores validated IANA time zone (e.g., `America/New_York`).
`event_types`	`id` (UUID PK), `host_id` (UUID FK), `name` , `slug` , `duration_minutes` , `capacity` , `buffer_before_minutes` , `buffer_after_minutes` , `minimum_notice_minutes` , `booking_horizon_days`	`UNIQUE(host_id, slug)` ; `CHECK (duration_minutes BETWEEN 15 AND 120)` ; `CHECK (capacity >= 1)` ; `CHECK (buffer_before_minutes BETWEEN 0 AND 30)` ; `CHECK (buffer_after_minutes BETWEEN 0 AND 30)` .
`weekly_availability`	`id` (UUID PK), `event_type_id` (UUID FK), `weekday` (INT), `start_time` (TIME), `end_time` (TIME)	`CHECK (weekday BETWEEN 1 AND 7)` (ISO 1=Mon..7=Sun); `CHECK (start_time < end_time)` . Multiple windows per day supported.
`date_overrides`	`id` (UUID PK), `event_type_id` (UUID FK), `local_date` (DATE), `is_unavailable` (BOOL), `start_time` (TIME), `end_time` (TIME)	`CHECK` enforces that `is_unavailable = true` has null times (full day off), whereas custom hours require `start_time < end_time` .

Table Name	Key Columns & Types	Integrity Constraints & Indexes
`bookings`	`id` (UUID PK), `event_type_id` (FK), `host_id` (FK), `status` (ENUM), `starts_at` (TIMESTAMPTZ), `ends_at` (TIMESTAMPTZ), `effective_starts_at`, `effective_ends_at`, `hold_expires_at`, `guest_name`, `guest_email`, `manage_token`	`status IN ('held', 'confirmed', 'cancelled', 'expired')`; `UNIQUE(manage_token)`; `CHECK (starts_at < ends_at)`; Partial indexes on `(host_id, effective_starts_at, effective_ends_at)` and `(event_type_id, starts_at, ends_at) WHERE status IN ('held', 'confirmed')`.
`idempotency_keys`	`key` (TEXT PK), `endpoint` (TEXT), `request_hash` (TEXT), `response_status` (INT), `response_body` (JSONB)	Primary key on `key`; stores SHA-256 hash of request parameters to prevent cross-payload key reuse (HTTP 422).
`notification_events`	`id` (UUID PK), `booking_id` (UUID FK), `event_type` (TEXT), `payload` (JSONB), `created_at` (TIMESTAMPTZ)	Audit & webhook sink table recording `booking.confirmed`, `booking.rescheduled`, and `booking.cancelled`.

4. Key Architectural Decisions & Alternatives Considered

Seven primary engineering decisions govern Slotbook's correctness, concurrency model, and developer experience.

Decision Area	Chosen Approach	Alternatives Considered	Why We Chose This Approach
1. Concurrency Control & Locking	Per-host transaction advisory locks (`pg_advisory_xact_lock`) paired with in-transaction availability verification.	(a) Pure `EXCLUDE USING gist` constraint; (b) `SERIALIZABLE` isolation with retry loops.	Pure GiST exclusion constraints cannot natively express group sessions (`capacity > 1`) where same-slot bookings are allowed up to `N` while cross-event buffer overlaps are forbidden. `pg_advisory_xact_lock` serializes both 1-on-1 and group capacity + buffer checks cleanly without serialization retry storms.

Decision Area	Chosen Approach	Alternatives Considered	Why We Chose This Approach
2. Hold Expiration Strategy	Hybrid SQL clock evaluation (`hold_expires_at > NOW()`) + lazy in-transaction status reaper.	Background cron job / polling worker running every minute to mark holds expired.	Background timers suffer from polling lag: a hold expiring at `10:05:00` might not be reaped until `10:05:59`, falsely blocking a slot. Filtering by `hold_expires_at > NOW()` in SQL frees capacity at the exact millisecond of expiry.
3. Time-Zone & DST Representation	Store bookings as UTC `TIMESTAMPTZ`; store schedules as local wall-clock `TIME` + IANA zone string; compute slots via Luxon in Host zone first.	Storing weekly availability as UTC offsets (e.g., `UTC-05:00`) or pre-generating slot rows in the database.	Fixed UTC offsets break twice a year when a host or guest crosses a Daylight Saving Time boundary. Generating slots dynamically from IANA rules keeps `09:00` anchored to the host's wall clock year-round.
4. Buffer Enforcement Model	Persist both nominal bounds (`starts_at`, `ends_at`) and buffered bounds (`effective_starts_at`, `effective_ends_at`) on each booking.	Recomputing buffers dynamically via SQL joins against `event_types` on every read query.	Storing effective bounds directly on `bookings` allows B-Tree partial index scans on `(host_id, effective_starts_at, effective_ends_at)` and ensures historical bookings keep the buffer rules active at the time they were booked.
5. Idempotency Storage	Dedicated `idempotency_keys` table storing SHA-256 payload hashes and JSON responses inside the booking transaction.	In-memory cache or simple `UNIQUE(idempotency_key)` column on `bookings` without payload hashing.	Storing the request hash detects client bugs where the same `Idempotency-Key` is accidentally reused for a different slot or event (returning HTTP 422 instead of silently returning the wrong hold).
6. Atomic Rescheduling	Single-transaction `UPDATE` on the existing booking row while excluding `booking.id` from the availability check.	Cancelling the old booking first and creating a new hold/booking in two separate requests.	A two-step cancel-then-book flow risks leaving the guest with no appointment if the new slot is taken mid-flight. An in-place transactional update is all-or-nothing and preserves the guest's `manage_token`.

Decision Area	Chosen Approach	Alternatives Considered	Why We Chose This Approach
7. Host Session Isolation on Public Routes	Read-only preview mode in UI + `x-host-session` guard on `POST /api/holds` when logged in as Host.	Allowing hosts to place holds on their own public booking pages while authenticated.	Prevents hosts from accidentally consuming their own capacity or placing 5-minute holds while previewing how their public calendar looks to guests.

5. Reliability Engineering & Failure Modes Matrix

The brief requires that nobody is ever double-booked—even when the network, the clock, or the user misbehaves. Below are the specific failure modes identified, how Slotbook prevents each, and the automated test or mechanism that proves it.

Failure Scenario	System Mitigation & Transactional Mechanism	Verification / Test Proof
1. Thundering Herd on 1-on-1 Slot (25 guests click the same slot at the same millisecond)	All 25 transactions call `pg_advisory_xact_lock(hashtext('host:' || hostId))`. PostgreSQL queues the 25 transactions sequentially. Transaction #1 sees 0 overlapping bookings and inserts the hold; Transactions #2–#25 see the new hold and abort with `409 SLOT_UNAVAILABLE`.	`tests/concurrency.test.ts` Test #1 (25 concurrent `createSlotHold` calls → 1 fulfilled, 24 rejected with 409).
2. Group Capacity Overflow (20 guests race for a `capacity = 3` session)	Inside the host advisory lock, `computeAvailableSlots` counts active rows matching `(event_type_id, starts_at, ends_at)`. Once 3 holds/confirmations exist, `remainingCapacity` reaches 0 and subsequent callers receive HTTP 409.	`tests/concurrency.test.ts` Test #2 (20 concurrent calls on `office-hours` → exactly 3 succeed, 17 fail with 409).
3. Cross-Event Buffer Collision (Booking adjacent slot that violates a 10m buffer)	Every booking stores `effective_starts_at = starts_at - buffer_before` and `effective_ends_at = ends_at + buffer_after`. Candidate slots are rejected if their buffered window overlaps any existing booking's nominal window or vice versa.	Verified in live booking flow: booking `10:00–10:30 EST` (10m buffers → `09:50–10:40`) automatically removes `09:30–10:00` and `10:30–11:00`.
4. Flaky Network & Double-Click Retries	`POST /api/holds` hashes `host:slug:slotStart` with SHA-256 and checks `idempotency_keys` inside the transaction lock. Retries with the same key return the original hold payload (HTTP 200) without decrementing capacity twice.	`tests/concurrency.test.ts` Test #3 (replaying `idem-lifecycle-key-1` returns identical `holdId` and `replayed: true`).
5. Abandoned Guest Holds & Clock Skew	All expiration timestamps use PostgreSQL's monotonic server clock (`NOW() + INTERVAL '5 minutes'`) rather than Node.js or browser clocks. Stale holds are ignored immediately by `hold_expires_at > NOW()` and reaped on the next transaction.	`tests/concurrency.test.ts` Test #4 (backdating `hold_expires_at` by 2s causes `confirmSlotHold` to throw `410 HOLD_EXPIRED` and frees the slot).

Failure Scenario	System Mitigation & Transactional Mechanism	Verification / Test Proof
6. Mid-Reschedule Race Condition	`rescheduleBookingByToken` locks the host, validates the target slot while passing `excludeBookingId` so the guest's current slot does not self-conflict, and updates the row atomically.	`tests/concurrency.test.ts` Test #3 (reschedules confirmed booking to `slots[1]` and verifies `booking.rescheduled` notification).
7. Invalid Guest Inputs & Host Self-Booking	Strict regex email validation (requiring `name@domain.tld`) on both React and Fastify Zod schemas; host session header check blocks holds while signed in as Host.	Verified in UI and `POST /api/holds` (returns `403 HOST_BOOKING_FORBIDDEN` when `x-host-session` is present).

6. Time-Zone Architecture & Worked Daylight-Saving Example

Time-zone handling is one of the most error-prone areas of scheduling software. Slotbook enforces a strict separation between **schedule definition time** (wall-clock time in the host's IANA time zone), **storage & conflict math** (UTC epoch milliseconds / PostgreSQL `TIMESTAMPTZ`), and **presentation time** (guest's selected IANA time zone).

How Slots Are Generated and Displayed

1. **Anchor to Database Clock:** `computeAvailableSlots` queries `SELECT NOW() AS now` from PostgreSQL and computes the valid booking window: `[max(requestFrom, dbNow + minimumNotice), min(requestTo, dbNow + bookingHorizon)]`.
2. **Day-by-Day Iteration in Host IANA Zone:** Using Luxon, the generator converts the start of the window into the host's IANA zone (e.g., `America/New_York`) and steps forward one local calendar day at a time.
3. **Override Resolution:** For each local date (`YYYY-MM-DD`), it checks `date_overrides`. If an override with `is_unavailable = true` exists, zero slots are generated for that date. If custom override hours exist, they replace the default `weekly_availability` windows for that weekday.
4. **Wall-Clock to UTC Conversion:** Each local window (e.g., `09:00` to `12:00`) is instantiated on that specific calendar date in the host's zone and stepped in increments of `duration_minutes`. Luxon resolves the exact IANA offset in effect for that date and converts each candidate slot to UTC (`slotStartUtc`, `slotEndUtc`) for database conflict checking, and to the guest's requested `tz` (`startLocal`, `endLocal`) for display.

Worked Daylight-Saving Time (DST) Transition Example

Consider our seeded demo host **Alex Rivera** (home time zone: `America/New_York`) who offers a 30-minute consultation (`intro-call`) Monday–Friday from `09:00` to `12:00` local New York time, and a guest browsing from **Hyderabad, India** (`Asia/Kolkata`, which does not observe DST and stays at `UTC+05:30` year-round).

In 2026, the United States ends Daylight Saving Time ("Fall Back") on **Sunday, November 1, 2026 at 02:00 AM**, shifting `America/New_York` from Eastern Daylight Time (**EDT, UTC−04:00**) to Eastern Standard Time (**EST, UTC−05:00**). Here is how Slotbook computes Alex's first morning slot (`09:00 AM` New York time) across that boundary:

Stage	Before DST Fallback (Friday, Oct 30, 2026 — EDT)	After DST Fallback (Monday, Nov 9, 2026 — EST)
1. Host Wall-Clock Rule	`2026-10-30T09:00:00` in `America/New_York`	`2026-11-09T09:00:00` in `America/New_York`
2. Active IANA Offset	`UTC−04:00` (Eastern Daylight Time)	`UTC−05:00` (Eastern Standard Time)
3. Stored UTC Timestamp (`starts_at`)	`2026-10-30T13:00:00.000Z` (09:00 + 4 hours)	`2026-11-09T14:00:00.000Z` (09:00 + 5 hours)
4. Guest Display in `Asia/Kolkata` (`UTC+05:30`)	`2026-10-30T18:30:00.000+05:30` Displayed as: **6:30 PM IST**	`2026-11-09T19:30:00.000+05:30` Displayed as: **7:30 PM IST**

> **Key Takeaway:** Because Slotbook anchors availability rules to the host's IANA zone (`America/New_York`) before converting to UTC and then to `Asia/Kolkata` , Alex's workday always begins at **9:00 AM New York time**, while the guest in India automatically sees the slot shift from **6:30 PM IST** (in October) to **7:30 PM IST** (in November).

7. Optional Extensions Built

Once the core booking engine and concurrency suite were verified, we implemented four optional extensions that integrate directly into the existing Fastify + PostgreSQL architecture:

- **1. Calendar Invites (`.ics` Attachment & Download Endpoint):** Implemented RFC 5545 iCalendar generation in `src/modules/notifications/service.ts` (`buildIcsCalendarInvite`). Every `booking.confirmed` , `booking.rescheduled` , and `booking.cancelled` event written to the notifications sink includes a complete `VEVENT` payload (with `METHOD:REQUEST` or `METHOD:CANCEL`), and guests can download the `.ics` file directly from their manage page via `GET /api/bookings/:token/invite.ics` .
- **2. Observability (Structured Request IDs, `/health` Endpoint & Live Metrics):** Configured Fastify with structured Pino JSON logging, correlation IDs (`reqId` / `x-request-id`), response-time tracking, and a real-time `GET /health` endpoint that queries PostgreSQL for live counts of `confirmedBookings` , `activeHolds` , `expiredHolds` , `cancelledBookings` , and `notificationsEmitted` .
- **3. Interactive OpenAPI / Swagger Documentation:** Registered `@fastify/swagger` and `@fastify/swagger-ui` at `GET /docs` so reviewers can inspect and test every endpoint directly in the browser.

- **4. Host Authentication & Role-Based Booking Guard:** Added username/password host login (`POST /api/host/login`), session token persistence, dynamic public navbar synchronization (`GET /api/hosts/:host/public-events`), and a role guard that switches public event pages into a read-only preview mode when the host is logged in.

8. Testing Coverage, Execution & Results

The automated test suite (`tests/concurrency.test.ts`) runs via Vitest against a real PostgreSQL 16 instance (never mocked). You can execute the entire suite with a single command:

```
cd slotbook-backend && npm test
```

Measured Test Suite Results (Real PostgreSQL 16 on CachyOS Linux)

Test Case Description	Assertions Verified	Execution Time
1. 1-on-1 Concurrency Race 25 simultaneous clients race for the same `intro-call` slot	1 HTTP 201 hold created; 24 rejected with HTTP 409; exactly 1 active row in PostgreSQL.	`471 ms` (PASS)
2. Group Session Capacity Race 20 simultaneous clients race for a `capacity = 3` `office-hours` slot	Exactly 3 HTTP 201 holds created; 17 rejected with HTTP 409; slot disappears from available list when full.	`147 ms` (PASS)
3. Idempotency & Full Lifecycle Replay `Idempotency-Key`, then `held → confirmed → rescheduled → cancelled`	Replay returns HTTP 200 with `replayed: true`; confirm generates `manageToken`; reschedule moves slot; 3 notifications + `.ics` emitted.	`99 ms` (PASS)
4. 5-Minute Hold Expiration Reclamation Expire an unconfirmed hold and verify capacity is freed	Late confirmation throws HTTP 410 (`HOLD_EXPIRED`); new guest immediately books the reclaimed slot.	`72 ms` (PASS)

Total Suite Runtime: 4 passed (4) in `986 ms` test execution time (`2.02 s` total wall time including TypeScript transform and schema setup).

9. Working with AI: What Helped, Where It Misled & Manual Corrections

AI coding assistance was used throughout development to accelerate boilerplate generation, draft Drizzle ORM schemas, and scaffold the React + Tailwind UI components. However, several subtle bugs and architectural edge cases required human diagnosis and manual engineering corrections:

- **1. Concurrent Queries on a Single `pg` Transaction Client (Where AI Misled):** The initial AI-generated implementation of `computeAvailableSlots` used `Promise.all([db.query(...), db.query(...), db.query(...)])` to fetch weekly availability, date overrides, and active bookings in parallel. While valid on a connection pool, passing a transactional `PoolClient` inside `withHostTransactionLock` triggered a Node.js `pg@8.x` deprecation warning (*"Calling client.query() when the client is already executing a query is deprecated and will be removed in pg@9.0"*) because a single PostgreSQL wire connection processes queries serially. We refactored those three queries to run sequentially when executing inside a transaction client.
- **2. Static Navbar Links vs. Dynamic Event Creation:** The initial frontend scaffold hardcoded links to the two seeded events (`/alex/intro-call` and `/alex/office-hours`). When we created a third event type in the Host Dashboard, it saved to PostgreSQL but did not appear in the top navigation bar. We added `GET /api/hosts/:host/public-events` and updated `App.tsx` to dynamically render all active host events.
- **3. HTML5 Email Validation Permissiveness:** Standard HTML5 `<input type="email">` accepts local-domain strings without a top-level domain (such as `nov25@fiehionefion`), which then failed against Zod's stricter backend validator with a generic error banner. We added explicit TLD regex validation (`/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/`) on both frontend and backend and improved Fastify's `ZodError` formatter to return field-specific messages.
- **4. Host Self-Booking Prevention:** During manual testing, we noticed that an authenticated host could navigate to their own public booking page and place holds on their own schedule. We engineered a Host View-Only mode on the public booking page and enforced an `x-host-session` guard on `POST /api/holds` .

10. Known Gaps & Next Steps (With Another Day)

Within the assignment timebox, we prioritized transactional correctness, zero double-bookings, IANA/DST accuracy, and a complete 3-surface UI. Given one additional engineering day, we would implement the following enhancements:

- **1. Waitlist Promotion Engine:** Allow guests to join a waitlist on full slots and automatically issue a time-limited 15-minute hold to the next waitlisted guest whenever a confirmed booking is cancelled.
- **2. Outbox Pattern & Signed HMAC Webhook Worker:** Add a background worker using `SELECT ... FOR UPDATE SKIP LOCKED` on `notification_events` to deliver signed HMAC-SHA256 webhooks with exponential backoff retries.
- **3. Granular Lock Sharding:** Currently, `pg_advisory_xact_lock` locks at the `host_id` level, which is ideal because cross-event buffers apply across a host's calendar. For high-throughput hosts, we could shard the advisory lock key to `hashtext(host_id || ':' || local_date)` so bookings on different days never contend on the same lock.
- **4. Playwright E2E Browser Suite & k6 Load Test Script:** Add automated headless browser tests for the React UI and a k6 load script benchmarking p95 latency at 500 RPS.